iMOVIES

MOVIE DISCOVERY & TICKET BOOKING WEB APPLICATION

Submitted by: Aastha Patidar

Technology Stack: React, Node.js, Express.js, MongoDB

Live Website: https://i-movies-chi.vercel.app/

GitHub Repository: https://github.com/AasthaPatidar907/iMovies

Backend API: https://imovies-eex6.onrender.com

1. Introduction

Project Title: iMovies — Movie Discovery and Ticket Booking Web Application

Team Member: Aastha Patidar — project development, frontend/backend integration, database connectivity, and deployment.

iMovies is a full-stack web application built to let users browse movies and manage a cinema ticket-booking workflow. It uses React for the frontend, Node.js and Express.js for backend API processing, and MongoDB for persistent data storage. The application includes account access, city/cinema selection, seat selection, wishlist management, and booking review.

2. Overview Project

Purpose

The purpose of iMovies is to provide a simple, interactive website for discovering movies and demonstrating a ticket-booking flow from movie browsing through seat selection and booking management.

Features

- User registration, login, and logout.
- Movie listings and browsing.
- City and cinema selection in the booking flow.
- Seat selection with categories such as Gold, Silver, Balcony, and Club where available in the interface.
- Wishlist functionality for saving and removing movies.
- Booking creation, viewing saved bookings, and booking cancellation/removal.
- Persistent data handled through the backend and MongoDB.
- Deployed frontend and backend services.

The project is an academic demonstration. A real payment gateway and official live inventory integration with cinema chains are not claimed as implemented.

3. Architecture

Frontend

The frontend is built with React and Vite. It renders the movie interface, account forms, city/cinema selection, seat-selection interface, wishlist, and booking panels. It communicates with the backend through HTTP requests and updates the interface based on API responses.

Backend

The backend uses Node.js and Express.js to expose API routes, process requests, validate inputs, and coordinate database operations. The deployed backend base URL is https://imovies-eex6.onrender.com.

Database

MongoDB Atlas is used for hosted NoSQL storage, with Mongoose supporting the database connection and data modelling. Logical data areas include user/account records, movie data, booking records, and wishlist items. Exact fields depend on the models in the current repository.

Architecture Flow

1. The user interacts with the React frontend.
2. The frontend sends a request to the Express API.
3. The backend processes the request and accesses MongoDB through Mongoose when required.
4. The backend returns a response.
5. The frontend updates the displayed data.

4. Setup Instructions

Prerequisites

- Node.js and npm.
- Git, if cloning the repository.
- MongoDB Atlas or another accessible MongoDB instance.
- Visual Studio Code or another code editor.

Installation

6. Clone the repository: git clone https://github.com/AasthaPatidar907/iMovies.git
7. Open the project folder and locate the frontend and backend directories containing their respective package.json files.

8. Run npm install in each application directory.
9. Create the backend .env file using the exact variable names required by the server code.
10. Configure the MongoDB connection URI, port, and JWT secret as required. Never share actual secret values or commit the .env file.
11. Confirm the database connection and deployment environment settings before starting the application.

Example environment variable names (use the names required by your own server code):

PORT=5000
MONGO_URI=<your MongoDB connection string>
JWT_SECRET=<your private JWT secret>

5. Folder Structure

The project separates the React frontend from the Express backend. Use the exact folder names from the final GitHub repository when presenting the structure.

Area	Purpose
Frontend	React pages/components, CSS, assets, routing, and API calls.
Backend	Express server, routes, controllers, models, middleware, and database connection.
Environment configuration	Local backend configuration; secrets should remain private.
package.json	Dependencies and scripts for each application.
GitHub repository	Version control and source-code hosting.

Before submission, verify that node_modules and private .env files are excluded from the public repository.

6. Running the Application

Frontend

Open a terminal in the frontend directory, install dependencies if needed, and run the script defined in package.json. For a Vite project this is commonly:

npm run dev

Backend

Open a second terminal in the backend directory and run the configured development or start script. For example, if the project defines it:

npm run dev

Run frontend and backend in separate terminals. Use the local URL shown by Vite for local testing. The deployed website can be opened directly at https://i-movies-chi.vercel.app/.

7. API Documentation

The frontend communicates with the backend through HTTP endpoints. The following endpoints are documented from the available project details; check the current route files before adding any other route.

Method	Endpoint	Purpose
GET	/api/health	Returns backend service/health status.
GET	/api/movies?city=Indore	Retrieves movie data for the selected city where supported.
POST	/api/bookings	Creates a booking; the request body must match the backend controller.
DELETE	/api/bookings/:bookingId?email=:email	Cancels or removes a booking using the booking identifier and email query parameter.

Base URL: https://imovies-eex6.onrender.com

Example URLs:

GET https://imovies-eex6.onrender.com/api/health

GET https://imovies-eex6.onrender.com/api/movies?city=Indore

Request bodies, authentication requirements, and response formats should be checked against the current source code. Do not invent undocumented API routes.

8. Authentication

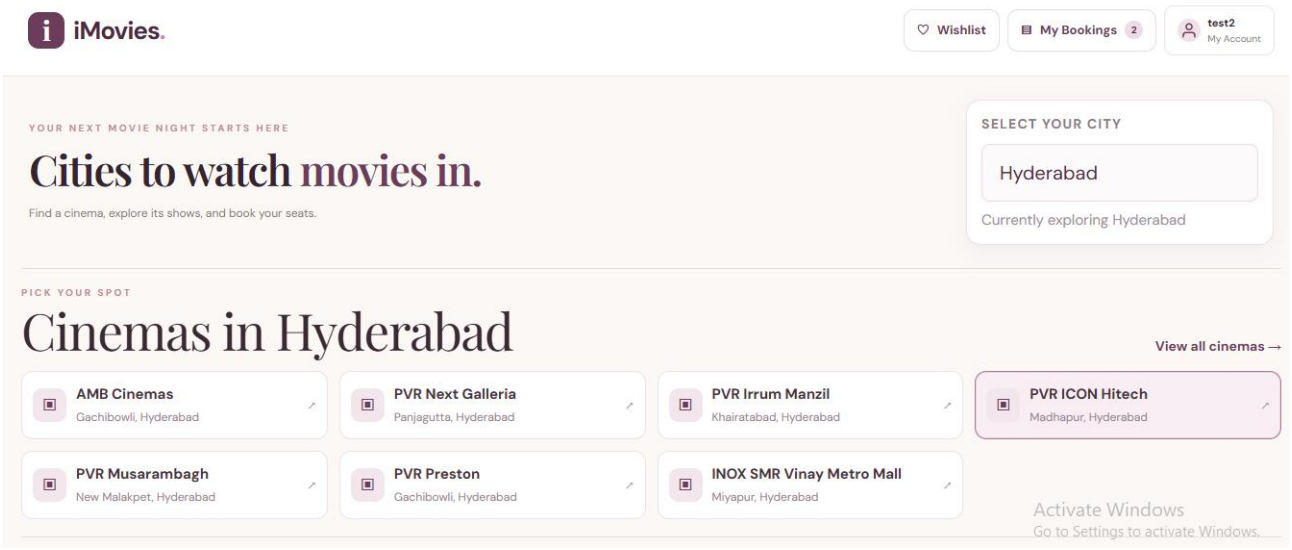
The application supports registration, login, and logout. The backend authentication flow uses JWT and password-hashing support through bcrypt/bcryptjs where configured. Credentials should be validated on the backend, and passwords should be stored as hashes rather than plain text.

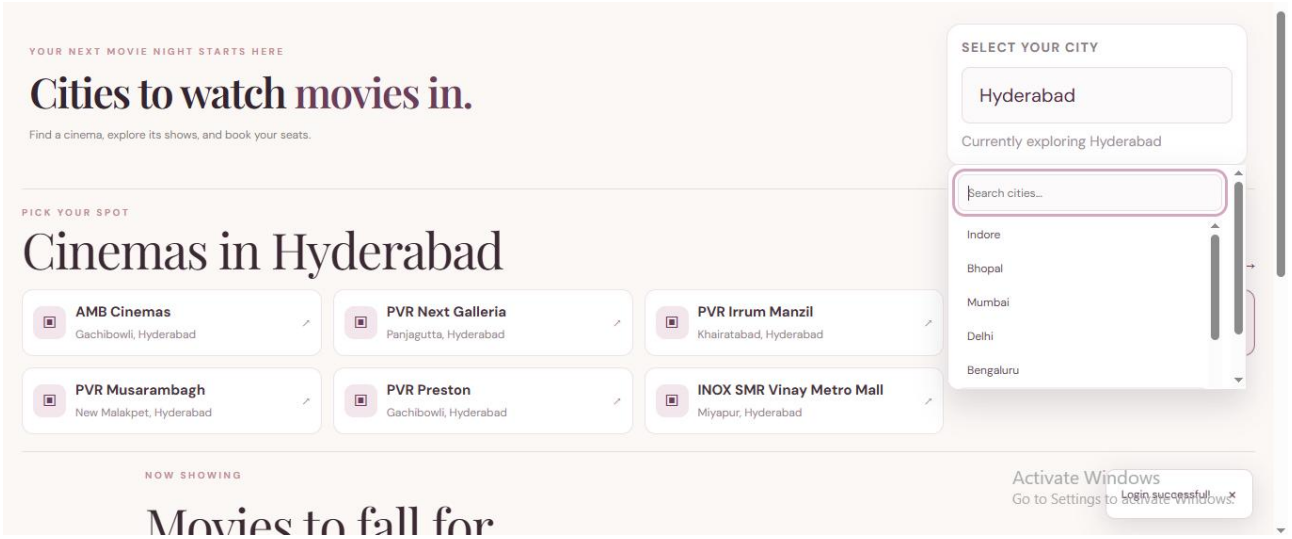
- Keep database connection strings and JWT secrets in environment variables.
- Never include passwords, tokens, or database credentials in screenshots or public documentation.
- Validate user-supplied values on the backend.
- Associate wishlist and booking operations with the correct active user.
- On logout, clear the active authentication state and user-specific information from the interface.

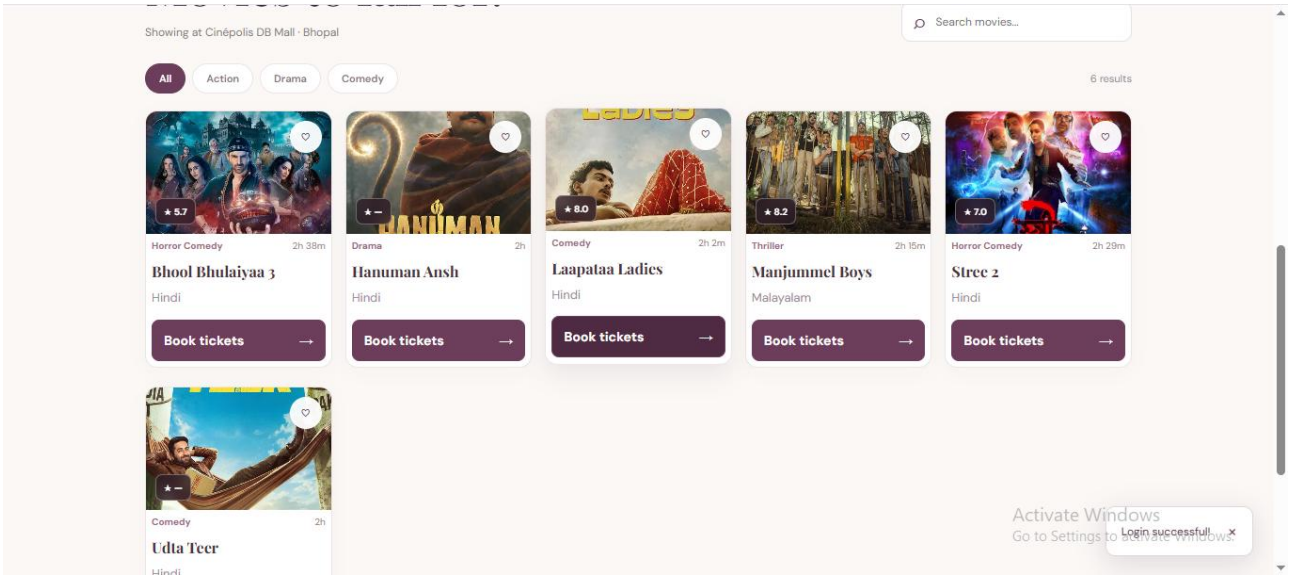
Exact token storage, token expiry, and protected-route details should be documented from the current source code rather than assumed.

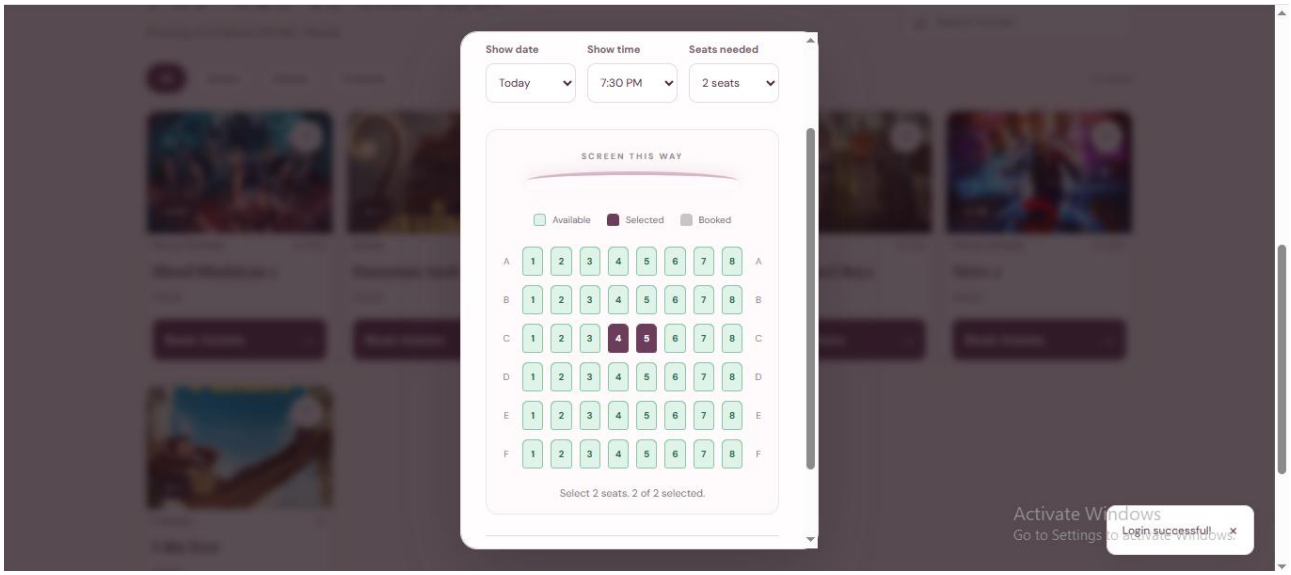
9. User Interface

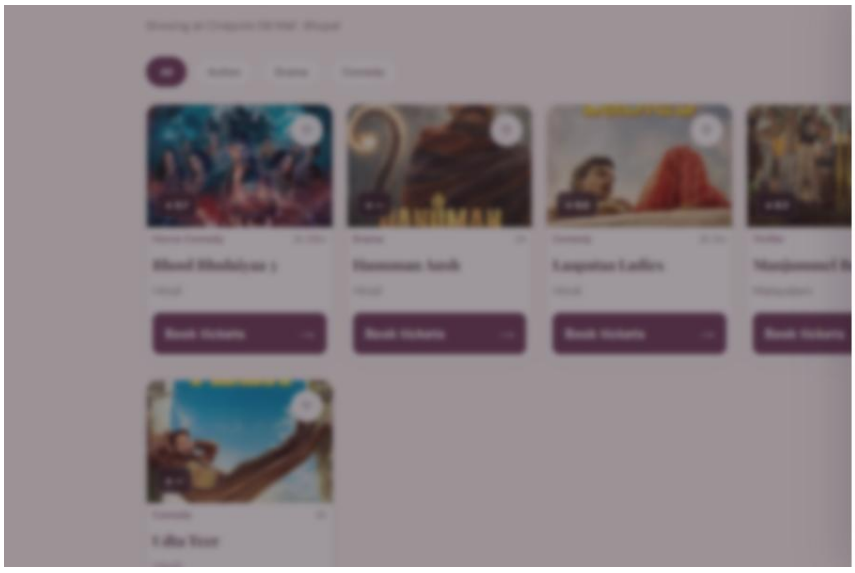
The user interface provides movie browsing and the main booking workflow. Add screenshots from the deployed website below. Capture the actual application at a readable zoom level and ensure that no passwords or private account details are visible.

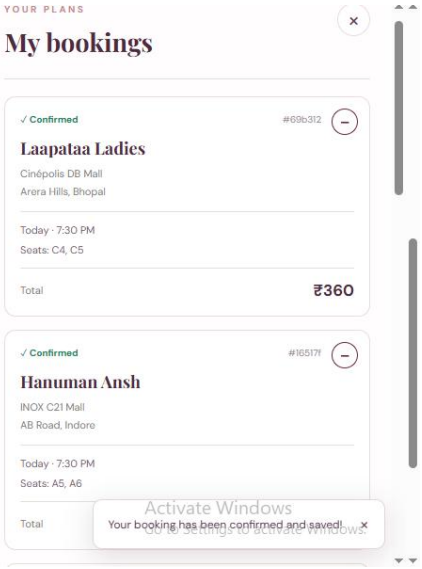

iMovies.
Wishlist
My Bookings 2
test2
My Account
YOUR NEXT MOVIE NIGHT STARTS HERE
Cities to watch movies in.
Find a cinema, explore its shows, and book your seats.
SELECT YOUR CITY
Hyderabad
Currently exploring Hyderabad
PICK YOUR SPOT
Cinemas in Hyderabad
View all cinemas →
AMB Cinemas
Gachibowli, Hyderabad
PVR Next Galleria
Panjagutta, Hyderabad
PVR Irrum Manzil
Khairatabad, Hyderabad
PVR ICON Hitech
Madhapur, Hyderabad
PVR Musarambagh
New Malakpet, Hyderabad
PVR Preston
Gachibowli, Hyderabad
INOX SMR Vinay Metro Mall
Miyapur, Hyderabad
Activate Windows
Go to Settings to activate Windows.


YOUR NEXT MOVIE NIGHT STARTS HERE
Cities to watch movies in.
Find a cinema, explore its shows, and book your seats.
SELECT YOUR CITY
Hyderabad
Currently exploring Hyderabad
Search cities...
Indore
Bhopal
Mumbai
Delhi
Bengaluru
PICK YOUR SPOT
Cinemas in Hyderabad
AMB Cinemas
Gachibowli, Hyderabad
PVR Next Galleria
Panjagutta, Hyderabad
PVR Irrum Manzil
Khairatabad, Hyderabad
PVR Musarambagh
New Malakpet, Hyderabad
PVR Preston
Gachibowli, Hyderabad
INOX SMR Vinay Metro Mall
Miyapur, Hyderabad
NOW SHOWING
Movies to fall for
Activate Windows
Go to Settings to activate Windows.
Login successful!

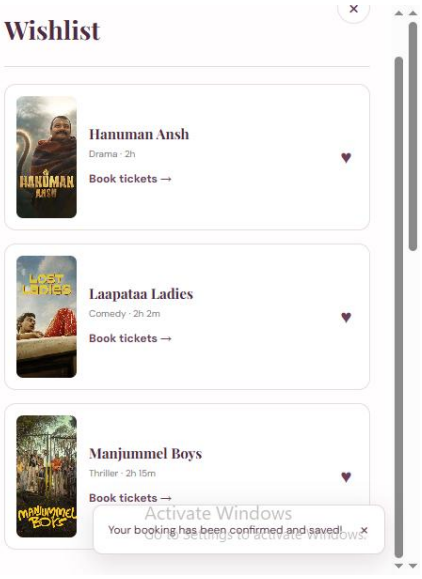

Showing at Cinépolis DB Mall · Bhopal
Search movies...
All
Action
Drama
Comedy
6 results
★ 5.7
Horror Comedy
2h 38m
Bhool Bhulaiyaa 3
Hindi
Book tickets
★ –
Drama
2h
Hanuman Ansh
Hindi
Book tickets
★ 8.0
Comedy
2h 2m
Laapataa Ladies
Hindi
Book tickets
★ 8.2
Thriller
2h 15m
Manjummel Boys
Malayalam
Book tickets
★ 7.0
Horror Comedy
2h 29m
Stree 2
Hindi
Book tickets
★ –
Comedy
2h
Udta Teer
Activate Windows
Go to Settings to activate Windows.
Login successful!


Show date
Show time
Seats needed
Today
7:30 PM
2 seats
SCREEN THIS WAY
Available
Selected
Booked
Select 2 seats. 2 of 2 selected.
Login successful!




YOUR PLANS
My bookings
✓ Confirmed
Laapataa Ladies
Cinépolis DB Mall
Arera Hills, Bhopal
Today · 7:30 PM
Seats: C4, C5
Total
₹360
✓ Confirmed
Hanuman Ansh
INOX C21 Mall
AB Road, Indore
Today · 7:30 PM
Seats: A5, A6
Total
Your booking has been confirmed and saved!


iMovies
Cities to watch movies in.
Cinemas in Bhopal
Movies to fall for.


Wishlist
Hanuman Ansh
Drama · 2h
Book tickets →
Laapataa Ladies
Comedy · 2h 2m
Book tickets →
Manjummel Boys
Thriller · 2h 15m
Book tickets →
Your booking has been confirmed and saved!

10. Testing

Testing should cover the main user journeys in the deployed application and, where applicable, the backend API. Mark the status only after running each test.

Test	Action	Expected result	Status
Frontend	Open the deployed website.	The iMovies interface loads.	Verify
Authentication	Register or log in with valid/invalid details.	Valid details follow the expected flow; invalid details are handled.	Verify
Movie browsing	Browse movie listings.	Movie cards and available information appear.	Verify
City/cinema	Change the city and inspect cinema choices.	The available choices update as implemented.	Verify
Seat booking	Select seats and complete the booking flow.	Booking details are reflected in the application.	Verify
Wishlist	Add/remove a movie and revisit the wishlist.	The displayed wishlist reflects the changes.	Verify
Bookings	Open My Bookings and refresh the page.	Saved booking information can be retrieved.	Verify
Logout/login	Log out and sign in again.	User-specific data corresponds to the active account.	Verify
Backend health	Open /api/health.	The endpoint returns a	Verify

		status response.	

Testing tools may include a web browser, browser developer tools, Postman, and GitHub. Do not claim automated test coverage unless automated test files and scripts exist.

11. Screenshots or Demo

Live website: https://i-movies-chi.vercel.app/

GitHub repository: https://github.com/AasthaPatidar907/iMovies

Demo video: [Paste your Google Drive share link here after uploading the video.]

For the Google Drive video, set General access to “Anyone with the link” and permission to “Viewer” if allowed by the submission rules. Test the link in an incognito window before submitting.

If the platform asks for a live website, GitHub repository, and demo video separately, submit each link in its corresponding field.

12. Known Issues

- The application is an academic/demo project and does not claim official cinema-chain inventory integration.
- A real payment gateway is not documented as implemented.
- Movie and cinema availability depends on the data and logic configured in the application.
- Authentication, wishlist, and booking behaviour should be rechecked after refresh, logout, and login to ensure the correct user's data is shown.
- Additional API routes should be listed only after checking the current source code.

Update this section if you identify a specific reproducible issue in the final deployed build. Remove any point that does not apply.

13. Future Enhancements

- Integrate a secure payment provider.
- Connect to reliable movie/cinema data for current showtimes and seat availability.
- Improve search, filtering, and sorting.

- Add booking confirmation notifications.
- Expand automated tests for authentication, wishlist, and booking APIs.
- Improve accessibility and responsiveness across mobile, tablet, and desktop screens.
- Add administrative tools for managing movies, cinemas, and show schedules.